

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	Deepan Kumar P	Reg. No.:	192312498
Section / Batch:	BE ECE 4 th Year	Date:	10/08/2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Analyze a processor datapath that performs register transfers and logical operations such as XOR, NOT, and comparison, and examine the control signals and execution sequence required for each operation.
2. Develop a bus-based system for transferring data between the processor, memory, and I/O devices, and analyze how bus width, transfer rate, and bus contention influence system performance.
3. Simulate a pipelined processor with arithmetic, memory, and branch instructions, and evaluate the effects of pipeline depth on execution time, throughput, and overall speedup.
4. Construct a pipeline containing instruction dependencies and branch conditions, and analyze the performance impact of data and control hazards using stalling, forwarding, and branch prediction techniques.
5. Develop a superscalar processor model that supports multiple execution units and out-of-order instruction processing, and evaluate how instruction scheduling and resource allocation improve instruction throughput.

Code of Conduct

I Deepan Kumar P (192312498) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

1. **Aim**

To perform register transfer, XOR, NOT, and comparison operations using the 8085 processor.

MVI A,55H

MOV B,A

MVI C,0FH

; XOR

MOV A,B

XRA C

STA 2000H

; NOT

MOV A,B

CMA

STA 2001

; Compare

MOV A,B

CMP C

JZ EQUAL

JC LESS

MVI D,02H

JMP STOP

EQUAL:

MVI D,00H

JMP STOP

LESS:

MVI D,01H

STOP:

HLT

OUTPUT

Register Transfer: B = 55H

XOR Result: 5AH

NOT Result: AAH

Comparison: A > C

2.Aim

To develop a bus-based system for transferring data between the **processor, memory, and I/O devices** and analyze the effect of bus width, transfer rate, and bus contention.

Code;

; Bus Data Transfer Simulation

MVI A,55H ; CPU sends data

STA 2000H ; Transfer data to memory

MVI A,0AH ; CPU sends data to I/O

OUT 01H ; Send data to I/O port

IN 01H ; Receive data from I/O

STA 2001H ; Store received data

HLT

OUTPUT;

Memory[2000H] = 55H

I/O Port 01H = 0AH

Received Data = stored at 2001H

3. Aim

To simulate a pipelined processor with arithmetic, memory, and branch instructions and evaluate the effect of pipeline depth on execution time, throughput, and speedup.

IF → ID → EX → MEM → WB

☐ **IF** – Instruction Fetch

☐ **ID** – Instruction Decode

☐ **EX** – Execute

□ **MEM** – Memory Access

□ **WB** – Write Back

Instructions:

I1: ADD R1,R2,R3

I2: SUB R4,R5,R6

I3: LOAD R7,[R8]

I4: BEQ R7,R0,L1

Cycle	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX
7			WB	MEM

Result

The pipelined processor improves instruction throughput by allowing multiple instructions to execute simultaneously in different stages.

4. Aim

To construct a pipeline containing instruction dependencies and branch conditions and analyze the performance impact of **data and control hazards** using stalling, forwarding, and branch prediction.

Instructions

I1: ADD R1,R2,R3

I2: SUB R4,R1,R5

I3: AND R6,R4,R7

I4: BEQ R6,R0,L1

Data Dependency

- I1 produces R1, which is required by I2.

- I2 produces R4, which is required by I3.

Therefore, **data hazards** occur.

Control Dependency

I4 is a branch instruction. The processor does not immediately know whether the branch will be taken, creating a **control hazard**.

Hazard Handling

1.Stalling:

The processor inserts empty cycles until the required data becomes available. This increases execution time.

2.Forwarding:

The result of an earlier instruction is directly forwarded to a dependent instruction, reducing unnecessary stalls.

3.BranchPrediction:

The processor predicts whether the branch is taken or not taken. Correct prediction reduces delay, while incorrect prediction causes pipeline flushing.

Result

Data and control hazards reduce pipeline performance by introducing stalls and delays. Forwarding reduces data hazards, while branch prediction reduces control-hazard penalties.

5. Aim

To develop a superscalar processor model with multiple execution units and out-of-order instruction processing, and evaluate how instruction scheduling and resource allocation improve instruction throughput.

Instruction:

I1: ADD R1,R2,R3

I2: LOAD R4,[R5]

I3: SUB R6,R7,R8

I4: ADD R9,R10,R11

Out-of-Order Execution

If I1 is delayed, ready instructions such as I2 and I3 can execute without waiting:

I1 → Waiting

I2 → Load Unit → Execute

I3 → ALU → Execute

This improves processor utilization.

Instruction Scheduling

The scheduler assigns each instruction to a suitable execution unit based on its operation and availability.

Resource Allocation

Multiple execution units allow several independent instructions to execute in the same clock cycle, reducing idle time.

Performance

For 8 independent instructions:

- Single execution unit = **8 cycles**
- Two execution units = **4 cycles**

Conclusion:

A superscalar processor improves performance by using multiple execution units and executing independent instructions simultaneously. Out-of-order execution and effective resource allocation increase instruction throughput and processor utilization.